

Docker & Kubernetes

Tutorial Completo Paso a Paso

Aprende a containerizar aplicaciones con Docker
y orquestarlas en producción con Kubernetes

Tutorial interactivo con ejemplos funcionales
para macOS, Linux y Windows

Generado: 09 de July de 2026

Tabla de Contenidos

1	Introducción: Conceptos de Docker y Kubernetes	3
2	Instalación en macOS	4
3	Instalación en Linux	4
4	Instalación en Windows	5
5	Ejemplo 1: Docker - Construir y ejecutar contenedor	6
6	Ejemplo 2: Kubernetes - Desplegar aplicación	7
7	Operaciones comunes (logs, escalar, monitoreo)	8
8	Compartir en GitHub	9

1. Introducción: Docker y Kubernetes

¿Qué es Docker?

Docker es una plataforma que permite empaquetar tu aplicación, dependencias y configuración en una unidad llamada 'contenedor'. Un contenedor es como una máquina virtual pero mucho más ligera.

Ventajas:

- Consistencia: Funciona igual en tu máquina, en testing y en producción
- Eficiencia: Usa menos recursos que VMs tradicionales
- Portabilidad: Puedes correr en cualquier máquina con Docker instalado
- Aislamiento: Cada contenedor está aislado de otros

¿Qué es Kubernetes?

Kubernetes (K8s) es un orquestador de contenedores. Gestiona automáticamente múltiples contenedores, distribuyendo carga, escalando, reiniciando en caso de fallo, etc.

Características principales:

- Auto-escalado: Aumenta/disminuye pods automáticamente
- Auto-recuperación: Reinicia contenedores que fallan
- Actualizaciones sin downtime: Rolling updates
- Balanceo de carga: Distribuye tráfico automáticamente

2. Instalación en macOS

Opción A: Docker Desktop (Recomendado)

1. Descarga Docker Desktop desde:

```
https://www.docker.com/products/docker-desktop
```

2. Ejecuta el instalador (.dmg) y sigue las instrucciones

3. Docker Desktop incluye: Docker Engine, Docker CLI, Docker Compose

4. También puedes activar Kubernetes desde Settings -> Kubernetes

Opción B: Homebrew

Si prefieres instalar por línea de comandos:

```
brew install docker docker-compose
```

Verificar instalación

Abre Terminal y ejecuta:

```
docker --version  
docker run hello-world
```

Si ves el mensaje 'Hello from Docker!', está correctamente instalado.

3. Instalación en Linux

Ubuntu/Debian

Paso 1: Actualizar paquetes del sistema

```
sudo apt update  
sudo apt upgrade -y
```

Paso 2: Instalar Docker desde repositorio oficial

```
curl -fsSL https://get.docker.com -o get-docker.sh  
sudo sh get-docker.sh
```

Paso 3: Agregar tu usuario al grupo docker (para no usar sudo)

```
sudo usermod -aG docker $USER  
newgrp docker
```

Paso 4: Verificar instalación

```
docker --version  
docker run hello-world
```

4. Instalación en Windows

Requisitos previos

- Windows 10 Pro, Enterprise o Education (No Home)
- Procesador compatible con virtualización
- Al menos 4 GB RAM disponible
- Hyper-V debe estar habilitado

Paso 1: Habilitar Hyper-V

Abre PowerShell como Administrador y ejecuta:

```
Enable-WindowsOptionalFeature -FeatureName Hyper-V -All
```

Reinicia la computadora cuando se pida

Paso 2: Instalar Docker Desktop

1. Descarga desde: <https://www.docker.com/products/docker-desktop>
2. Ejecuta el instalador
3. Marca 'Install required Windows components for WSL 2'
4. Reinicia al terminar

Paso 3: Verificar

Abre PowerShell o CMD y ejecuta:

```
docker --version  
docker run hello-world
```

5. Ejemplo 1: Docker - Contenedor funcional

¿Qué haremos?

Construiremos una imagen Docker de una aplicación web con Flask, la ejecutaremos en un contenedor, y la veremos funcionar en el navegador.

Paso 1: Crear estructura de archivos

En terminal macOS (dentro del proyecto):

```
cd /Users/daniel/curso_git_dpl/curso_git_diplomado
mkdir -p src/docker_kubernetes_tutorial
```

Paso 2: Crear archivos necesarios

Archivo 1: Dockerfile (plantilla para imagen)

```
# Ver archivo: Dockerfile
# Contiene instrucciones para construir la imagen
```

Archivo 2: app.py (aplicación web)

```
# Ver archivo: src/docker_kubernetes_tutorial/app.py
# Aplicación Flask que muestra página HTML
```

5 (cont). Construir y ejecutar contenedor Docker

Paso 3: Construir la imagen Docker

En terminal, desde la raíz del proyecto:

```
cd /Users/daniel/curso_git_dpl/curso_git_diplomado
docker build -t mi-app-web:latest .
```

Explicación del comando:

- docker build: Construye una imagen
- -t mi-app-web:latest: Tag (nombre:versión)
- .: Busca Dockerfile en directorio actual

Esto puede tardar 1-2 minutos la primera vez.

Paso 4: Ejecutar el contenedor

Una vez construida la imagen:

```
docker run -p 8080:5000 mi-app-web:latest
```

Explicación:

- docker run: Ejecuta un contenedor
- -p 8080:5000: Mapea puerto 8080 (local) -> 5000 (contenedor)
- mi-app-web:latest: Imagen a usar

Deberías ver en terminal:

```
'Running on http://0.0.0.0:5000'
```

Paso 5: Prueba en navegador

Abre tu navegador y ve a:

```
http://localhost:8080
```

Deberías ver una página web hermosa con:

- Título: 'Docker & Kubernetes Tutorial'
- Estado: '[OK] Contenedor ejecutándose correctamente'
- Stack tecnológico: Docker, Kubernetes, Python, Flask

¡Felicidades! Tu primer contenedor está funcionando.

5 (cont). Verificar y detener contenedor

Paso 6: Inspeccionar el contenedor

En otra ventana de terminal (sin cerrar la anterior):

Ver contenedores ejecutándose:

```
docker ps
```

Deberías ver algo como:

CONTAINER ID	IMAGE	STATUS	PORTS
abc123def456	mi-app-web...	Up 2 minutes	0.0.0.0:8080->5000/tcp

Paso 7: Ver logs

```
docker logs <container-id>
```

O si quieres ver logs en tiempo real mientras usas la app:

```
docker logs -f <container-id>
```

Paso 8: Detener el contenedor

Cuando termines, presiona Ctrl+C en la terminal donde corre Docker

O desde otra terminal:

```
docker stop <container-id>
```

6. Ejemplo 2: Kubernetes - Orquestrar contenedores

¿Qué haremos?

Usaremos la misma imagen Docker anterior, pero ahora la desplegaremos en Kubernetes. K8s creará automáticamente 3 copias (replicas) de la aplicación, con balanceo de carga.

Paso 1: Habilitar Kubernetes en Docker Desktop

Opción A (Docker Desktop con K8s):

1. Abre Docker Desktop
2. Ve a Settings -> Kubernetes
3. Marca 'Enable Kubernetes'
4. Espera 2-3 minutos a que inicie

Opción B (Minikube):

```
brew install minikube  
minikube start
```

Paso 2: Verificar K8s está corriendo

```
kubectl version --client  
kubectl get nodes
```

Deberías ver tu node (máquina local) en estado 'Ready'

6 (cont). Desplegar en Kubernetes

Paso 3: Preparar manifiestos de Kubernetes

Archivo: src/docker_kubernetes_tutorial/kubernetes.yaml

Este archivo define:

- Namespace: Separación lógica de recursos
- Deployment: 3 replicas de la aplicación
- Service: Expone la app en la red (puerto 8080)
- HPA: Auto-escalado basado en CPU

Paso 4: Aplicar manifiestos

```
kubectl apply -f src/docker_kubernetes_tutorial/kubernetes.yaml
```

Deberías ver:

```
namespace/docker-kubernetes-tutorial created
deployment.apps/mi-app-web created
service/mi-app-web created
horizontalpodautoscaler.autoscaling/mi-app-web-hpa created
```

Paso 5: Verificar que está corriendo

Ver deployments:

```
kubectl get deployments -n docker-kubernetes-tutorial
```

Ver pods (instancias de la aplicación):

```
kubectl get pods -n docker-kubernetes-tutorial
```

Deberías ver 3 pods en estado 'Running'

6 (cont). Acceder a la aplicación en Kubernetes

Paso 6: Port-forward (acceso local)

Para acceder a la aplicación desde tu navegador:

```
kubectl port-forward -n docker-kubernetes-tutorial \
  service/mi-app-web 8080:8080
```

Verás: 'Forwarding from 127.0.0.1:8080 -> 8080'

Paso 7: Acceder en navegador

```
http://localhost:8080
```

¡Verás la MISMA página web que en Docker!

Pero ahora está ejecutándose en Kubernetes, con:

- 3 replicas de la aplicación
- Balanceo de carga automático
- Auto-recuperación si un pod falla
- Auto-escalado si aumenta la demanda

7. Operaciones comunes en Kubernetes

Ver logs de un pod

```
kubectl logs <pod-name> \
-n docker-kubernetes-tutorial
```

Seguir logs en tiempo real

```
kubectl logs -f <pod-name> \
-n docker-kubernetes-tutorial
```

Describir pod (información detallada)

```
kubectl describe pod <pod-name> \
-n docker-kubernetes-tutorial
```

Escalar manualmente (cambiar replicas)

```
kubectl scale deployment/mi-app-web \
--replicas=5 \
-n docker-kubernetes-tutorial
```

Ver recursos de un pod

```
kubectl top pods -n docker-kubernetes-tutorial
```

8. Compartir en GitHub (paso a paso)

Paso 1: Ver cambios realizados

En terminal, verifica qué archivos cambiaron:

```
cd /Users/daniel/curso_git_dpl/curso_git_diplomado
git status
```

Deberías ver archivos nuevos (Dockerfile, app.py, etc.)

Paso 2: Agregar archivos al staging

Agrega todos los cambios del tutorial Docker/Kubernetes:

```
git add src/docker_kubernetes_tutorial/
git add Dockerfile
git add run_tutorial.sh
git add doc/Tutorial_Docker_Kubernetes.pptx
git add doc/Tutorial_Docker_Kubernetes_Paso_a_Paso.pdf
```

O más simple: agrega todo

```
git add -A
```

Paso 3: Verificar cambios a commit

```
git status
```

Deberías ver los archivos en verde (listos para commit)

Paso 4: Hacer commit

Crea un commit con un mensaje descriptivo:

```
git commit -m "feat: agregar tutorial completo de Docker y Kubernetes"
```

Mensaje incluye:

- feat: = feature (nueva funcionalidad)
- Descripción clara de qué se agregó

Paso 5: Enviar a GitHub

```
git push origin main
```

Verás:

```
Total X objects written
main -> main
```

Esto significa que los cambios están en GitHub

Paso 6: Verificar en GitHub

1. Abre navegador: https://github.com/danielxxi/curso_git_diplomado
2. Deberías ver los archivos nuevos
3. Los PDFs y PPTX se ven en la interfaz web

¡Felicidades!

Ya dominas Docker y Kubernetes.

Ahora puedes:

- [OK] Containerizar aplicaciones
- [OK] Ejecutar contenedores localmente
- [OK] Orquestar en Kubernetes
- [OK] Escalar automáticamente
- [OK] Monitorear y troubleshootear

Próximos pasos:

- Desplegar en producción (AWS, Azure, GCP)
 - Implementar CI/CD con Docker
 - Usar Helm para gestionar aplicaciones
- Implementar observabilidad (Prometheus, Grafana)